Short Report: Market Mechanics and Design Choices

Market Mechanics

The simulator operates as a **Continuous Double Auction (CDA)**. The primary mechanics involve:

- **The Spread:** The "gap" between the best bid and best ask. The simulator tracks the mid_price and spread_history, which naturally fluctuates based on the aggressiveness of the traders.
- **Liquidity Consumption:** Market orders "eat" through the book levels, causing **Market Impact**. This is a critical design choice—large orders in the simulator move the price, just as they do in reality.
- **Inventory Risk:** Market Makers are not "infinite." They have limited capital and inventory_target constraints, forcing them to widen spreads during high volatility to compensate for risk.

Design Choices

1. **Heap-Based Priority Queues:** We chose heapq for the LOB implementation. While a B-Tree or Skiplist is sometimes used for sorted access, heaps provide the fastest possible access to the "Top of Book" (L1 data), which is where 90% of matching activity occurs.
2. **Modular Agent Factory:** As shown in the **Component Diagram**, we implemented an Agent Factory. This allows us to "hot-swap" trading strategies (e.g., adding a Reinforcement Learning Agent) without modifying the core matching engine logic.
3. **Layered Architecture:** The separation of the Core Engine Layer from the Visualization Layer ensures that the simulation can run at maximum speed in "headless" mode for backtesting, only generating plots (matplotlib) when requested for human analysis.
4. **Redis Cache Integration:** For the physical deployment, the choice of a **Redis Cache** to store the current state of the Order Book allows for sub-millisecond lookups, mimicking the memory-resident databases used in high-frequency trading environments.
5. **Statistical Benchmarking:** The inclusion of kurtosis and skewness analysis in the analyze_market_quality function allows us to validate that the simulation is producing "Fat Tails": the statistical signature of real financial markets.